

Input/Output on WinMIPS64

September 29, 2025

Overview

WinMIPS64 exposes input/output through *memory-mapped* I/O (**MMIO**). Two special addresses are used:

- **CONTROL** at 0x10000
- **DATA** at 0x10008

To perform I/O, a program writes a *control code* to **CONTROL**. The **DATA** location holds the payload:

- For numbers: write or read the value directly at **DATA**.
- For strings: write the *address* (pointer) of a zero-terminated string to **DATA**.
- For pixel graphics (not required for the lab): use a specific packing of color and coordinates.

Common CONTROL Codes

Code	Effect
1	Output unsigned integer from DATA
2	Output signed integer from DATA
3	Output floating-point (double) from DATA
4	Output zero-terminated string; DATA = address
5	Graphics pixel: DATA =RGB, DATA +4=y, DATA +5=x
6	Clear terminal
7	Clear graphics display
8	Read numeric value (integer/float) into DATA
9	Read one byte (no echo) into DATA

Notes.

- Code 8 collects a numeric value: you type a number and press **Enter**; WinMIPS64 stores the parsed value at **DATA**.
- Code 9 returns a single raw byte. To read a line, loop with code 9 until you see '**\n**' and store bytes into a buffer.

Example: Read a Number and Echo it

The following program:

1. Prints a prompt (CONTROL = 4).
2. Reads a 64-bit integer (CONTROL = 8).
3. Prints it back as a signed integer (CONTROL = 2).

Assembler (MIPS64)

```
.data
CONTROL: .word32 0x10000
DATA: .word32 0x10008

prompt: .asciiz "Enter a number and press Enter:\n"
nl: .asciiz "\n"

.text

; Load MMIO register addresses
lwu $t9, CONTROL($zero) ; $t9 = CONTROL address
lwu $t8, DATA($zero) ; $t8 = DATA address

; Print prompt (CONTROL = 4: print string at address in DATA)
daddi $v0, $zero, 4
daddi $a0, $zero, prompt
sd $a0, 0($t8)
sd $v0, 0($t9)

; Read number (CONTROL = 8: read integer/float into DATA)
daddi $v0, $zero, 8
sd $v0, 0($t9)
ld $s0, 0($t8) ; $s0 = 64-bit integer read

; Echo as signed integer (CONTROL = 2)
daddi $v0, $zero, 2
sd $s0, 0($t8)
sd $v0, 0($t9)

; Newline (CONTROL = 4)
daddi $v0, $zero, 4
daddi $a0, $zero, nl
sd $a0, 0($t8)
sd $v0, 0($t9)

halt
```

The instructions here are very similar to that of MIPS, the only different being these instructions have doubleword versions as well (for 64 bit). So, `daddi` stands for doubleword addi, `sd` stands for store double word, etc. `halt` is not part of the MIPS64 ISA. It's a simulator-specific instruction that stops execution (like an `exit` `syscall` would on a real OS).

Single-Key Input

To wait for a single key without echo, use `CONTROL = 9`:

```
daddi $v0, $zero, 9 ; request a byte
sd $v0, 0($t9)
ld $t0, 0($t8) ; read from DATA
andi $t0, $t0, 0xFF ; keep just the low byte (the character)
```

This is useful for simple “*press any key to continue*” pauses or for building your own input routine byte-by-byte.